



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2022/2023

Computação Física
Relatório: Trabalho Prático 2

Turma: 22D	Grupo: 6
Nome: Miguel Alcobia	Número: 50746
Nome: Fábio Pestana	Número: 50756
Nome: Rafael Dias	Número: 50773
Professor: Jorge Pais	

Data: 4 de Maio de 2023

Índice

Lista de figuras	II
Lista de tabelas	III
Abreviaturas e símbolos.....	IV
Objetivos.....	1
Exercício 1	2
Enunciado:	2
1. 1 - Especificação da quantidade de bits de cada um dos registos.....	2
1. 2 - Quantidade de bits dos Address Bus e Data Bus das memórias de dados e de código.....	3
1. 3 - Codificação das instruções com o menor número de bits	4
1. 4 - Desenho do Módulo Funcional.....	4
1. 5 - Definição das entradas e saídas do Módulo de Controlo	9
1. 6 - Tabela de Verdade que implementa o Módulo de Controlo	12
1. 7 - Tabela de programação numa ROM que implementa o Módulo de Controlo	13
1.8 - Programas de Teste do CPU.....	14
Bibliografia.....	16
Código Comentado	17
Exercício 1:	17

Lista de figuras

Imagem 1 - Esquema da arquitetura de Harvard.....	3
Imagem 2 - Desenho dos registos.....	4
Imagem 3 - MOV V, const8, V = const8.....	5
Imagem 4 - MOV R, const6, R = const6.....	5
Imagem 5 - MOV V, A, V = A.....	6
Imagem 6 - MOV A, V, A = V.....	6
Imagem 7 - MOV V, @R, V = MD (R).....	7
Imagem 8 - MOV @R, V MD (R) = V.....	7
Imagem 9 - ADDC V, A V = V + A + C.....	8
Imagem 10 - NAND V, A, V = V nand A.....	8
Imagem 11 - JC end6, Se (C) PC = end6.....	9
Imagem 12 - JZ end6, Se (Z) PC = end6.....	9
Imagem 13 - Módulo Funcional com as entradas definidas.....	10
Imagem 14 - Esquema com os dois módulos distintos que formam o MC.....	11

Lista de tabelas

Tabela 1 - Tabela apresentada no enunciado.....	2
Tabela 2 - Funcionalidade e descrição de cada instrução.....	3
Tabela 3 - Codificação das instruções.....	4
Tabela 4 - Codificação das instruções com os valores variantes destacados.....	10
Tabela 5 - Tabela de verdade do Módulo de Controlo.....	12
Tabela 6 - Tabela de verdade do Módulo de Controlo com a Gama de Endereços definida.....	13
Tabela 7 - Elaboração do Programa 1.....	14
Tabela 8 - Elaboração do Programa 2.....	14
Tabela 9 - Elaboração do Programa 3.....	15

Abreviaturas e símbolos

Lista de abreviaturas

RAM	Random-access memory
ROM	Read-only memory
WE/	Write
RD/	Read
U.A.L.	Unidade Aritmética e Lógica

Lista de símbolos

+	Disjunção (OR)
.	Conjunção (AND)

Objetivos

Neste trabalho, os alunos tinham como tarefa projetar, construir e esquematizar um microprocessador ARV, baseado numa arquitetura de Harvard.

Para realizar esta atividade os alunos tiveram de inicialmente compreender as instruções dadas no enunciado, especificar a quantidade de bits de cada Registo, indicar o número de bits do *Address Bus* e do *Data Bus* nas respetivas memórias de dados e código, RAM e ROM respetivamente. Codificar instruções de forma a distingui-las (a codificação das instruções permite-nos obter o valor de bits do Data Bus da memória de código, dependendo da codificação este valor pode variar, complicando ou simplificando o projeto), desenhar um Módulo Funcional, definir os sinais de entrada e saída do Módulo de Controlo, realizar a tabela de verdade do Módulo de Controlo e implementar a tabela com uma ROM.

Para simular este projeto, os alunos desenvolveram e implementaram código para o Arduino, e criaram programas para testar o microprocessador e o seu funcionamento.

Exercício 1

Enunciado:

“O microprocessador tem os registos internos *V* (Valor), *R* (Referência) e *A* (Auxiliar), o registo de controlo de execução *PC* e as flags *C* (Carry) e *Z* (Zero). A abreviatura *MD* significa memória de dados. A abreviatura *const8* representa uma constante a 8 bits, a abreviatura *const6* representa uma constante a 6 bits e a abreviatura *end6* representa um valor absoluto a 6 bits.

O microprocessador ARV tem o seguinte conjunto de instruções: ”

Instrução	Funcionalidade
MOV V, const8	V = const8
MOV R, const6	R = const6
MOV V, A	V = A
MOV A, V	A = V
MOV V, @R	V = MD(R)
MOV @R, V	MD(R) = V
ADDC V, A	V = V + A + C
NAND V, A	V = V nand A
JC end6	Se (C) PC = end6
JZ end6	Se (Z) PC = end6
JMP end6	PC = end6

Tabela 1 - Tabela apresentada no enunciado

- Carvalho, Carlos; Niehus, Manfred & Pais, Jorge. (2023). *Computação Física 2.º Trabalho Prático*

1. 1 - Especificação da quantidade de bits de cada um dos registos

Foi pedido pelo professor que antes de iniciar-se o trabalho, os alunos explicassem pelas suas palavras o que cada função fazia. Consideramos este passo importante, pois ajuda a compreender o porquê do tamanho de cada registo. Segue a tabela elaborada pelos alunos:

Instrução	Funcionalidade	Descrição
MOV V, const8	V = const8	O registo V adquire o valor binário de 8 bits (a definir pelo programador)
MOV R, const6	R = const6	O registo R adquire o valor binário de 6 bits (a definir pelo programador)
MOV V, A	V = A	O registo V adquire o valor de A
MOV A, V	A = V	O registo A adquire o valor do registo V
MOV V, @R	V = MD(R)	O registo V adquire o conteúdo do endereço da memória de dados com o valor de R
MOV @R, V	MD(R) = V	O conteúdo do endereço da memória de dados definido pelo valor de R adquire o valor de V
ADDC V, A	V = V + A + C	O registo adquire o valor da soma (V + A + C)
NAND V, A	V = V nand A	O registo V adquire o valor da função lógica nand (V nand A)

JC end6	Se (C) PC = end6	Se C==1, então o registo PC assume o valor binário 6 bits (a definir pelo programador)
JZ end6	Se (Z) PC = end6	Se Zero==1, então o registo PC assume o valor binário 6 bits (a definir pelo programador)
JMP end6	PC = end6	Se o registo de controlo de execução, PC, fica com o valor binário (a definir pelo programador)

Tabela 2 - Funcionalidade e descrição de cada instrução

V - Registo de **8 bits**, pois V armazena informação de um valor binário a 8 bits (const8, V=const8)

A - Registo de **8 bits**, pois A armazena a informação de V, sendo que V é um valor binário a 8 bits.

R - Registo de **6 bits**, pois R armazena informação de um valor binário a 6 bits (const6, R=const6)

C - *Flag* do Carry, algoritmo do “resto/levantamento”, logo tem **1 bit** como dimensão.

Z - *Flag* do Zero, apresenta o 0, logo tem é um registo **1 bit**.

Const8 - Valor binário a **8 bits** (a definir pelo programador).

Const6 - Valor binário a **6 bits** (a definir pelo programador).

End6 - Valor binário a **6 bits** (a definir pelo programador).

PC - Registo de **6 bits**, pois armazena sempre o valor de end6.

1. 2 - Quantidade de bits dos Address Bus e Data Bus das memórias de dados e de código

Para uma melhor compreensão do porquê da quantidade de bits definida para cada Address Bus e Data Bus, segue uma ilustração da arquitetura de Harvard:

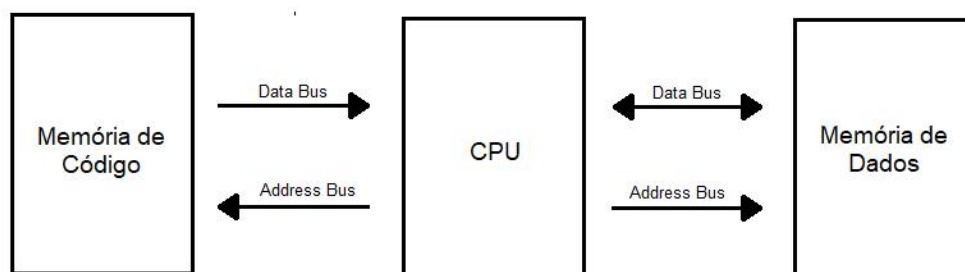


Imagem 1 - Esquema da arquitetura de Harvard

Memória de Dados:

Address Bus - **6 bits**, na funcionalidade $MD(R)=V$ demonstra a necessidade de 6bits devido a R a ser um registo a 6 bits.

Data Bus - **8 bits**, pois em $MD(R)=V$ e $V=MD(R)$ demonstra a necessidade de 8bits devido a V a ser um registo a 8 bits.

Memória de Código:

Address Bus - **6 bits**, pois o registo de controlo de execução (Program-Counter (PC)) assume a

valor de end6.

Data Bus - **x bits**, porque depende da codificação das instruções.

1.3 - Codificação das instruções com o menor número de bits

Instrução	Parâmetros	Codificação a 10 bits									
		D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
MOV V, const8	const8	0	0	C7	C6	C5	C4	C3	C2	C1	C0
MOV R, const6	const6	0	1	0	0	K5	K4	K3	K2	K1	K0
MOV V, A		0	1	0	1	0	0	0	0	0	0
MOV A, V		0	1	1	0	0	0	0	0	0	0
MOV V, @R		0	1	1	1	0	0	0	0	0	0
MOV @R, V		1	0	0	0	0	0	0	0	0	0
ADDC V, A		1	0	0	1	0	0	0	0	0	0
NAND V, A		1	0	1	0	0	0	0	0	0	0
JC end6	end6	1	0	1	1	E5	E4	E3	E2	E1	E0
JZ end6	end6	1	1	0	0	E5	E4	E3	E2	E1	E0
JMP end6	end6	1	1	0	1	E5	E4	E3	E2	E1	E0

Tabela 3 - Codificação das instruções

Tamanho Data Bus - 10 bits.

1.4 - Desenho do Módulo Funcional

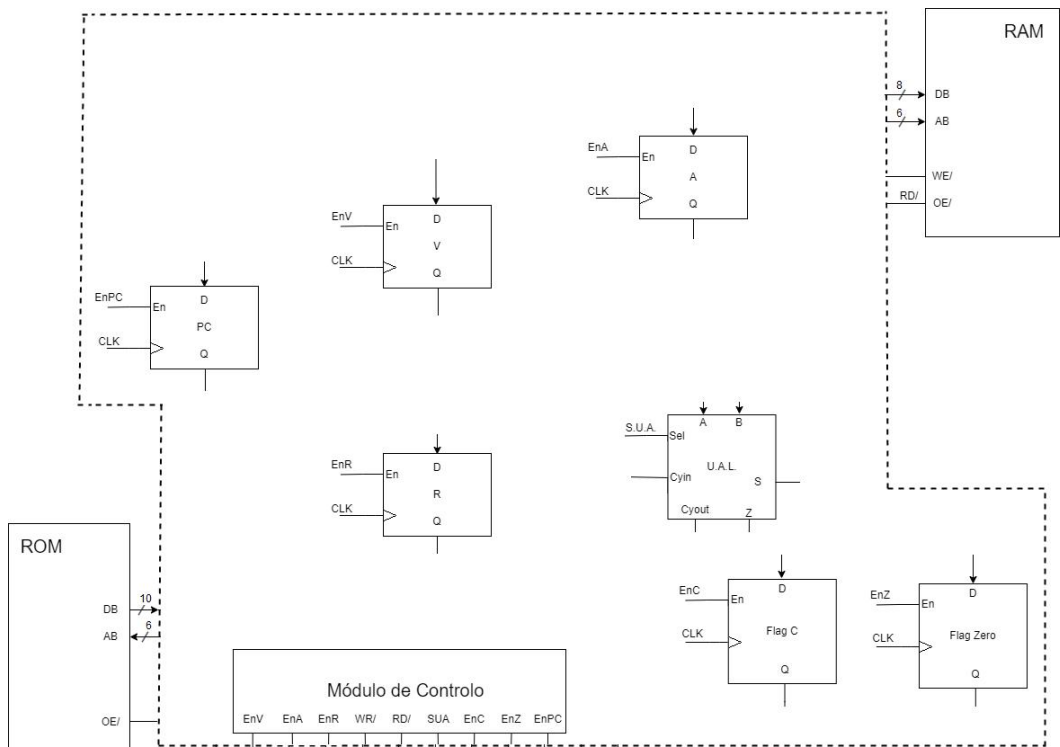


Imagem 2 - Desenho dos registros

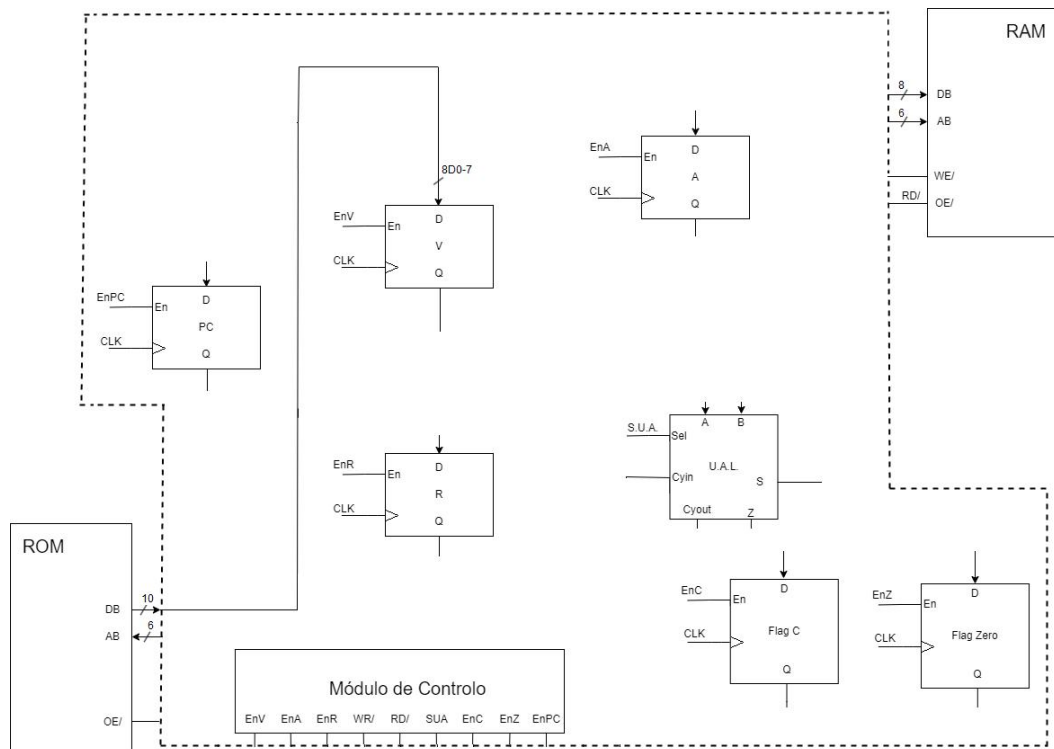


Imagem 3 - MOV V, const8, V = const8

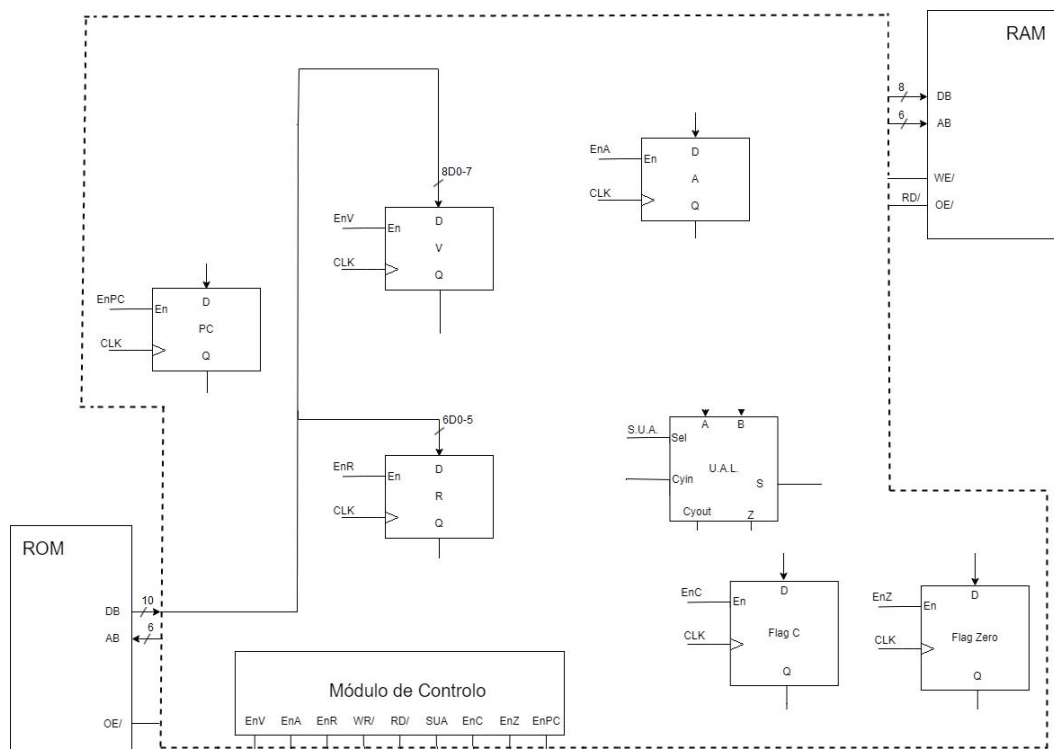


Imagem 4 - MOV R, const6, R = const6

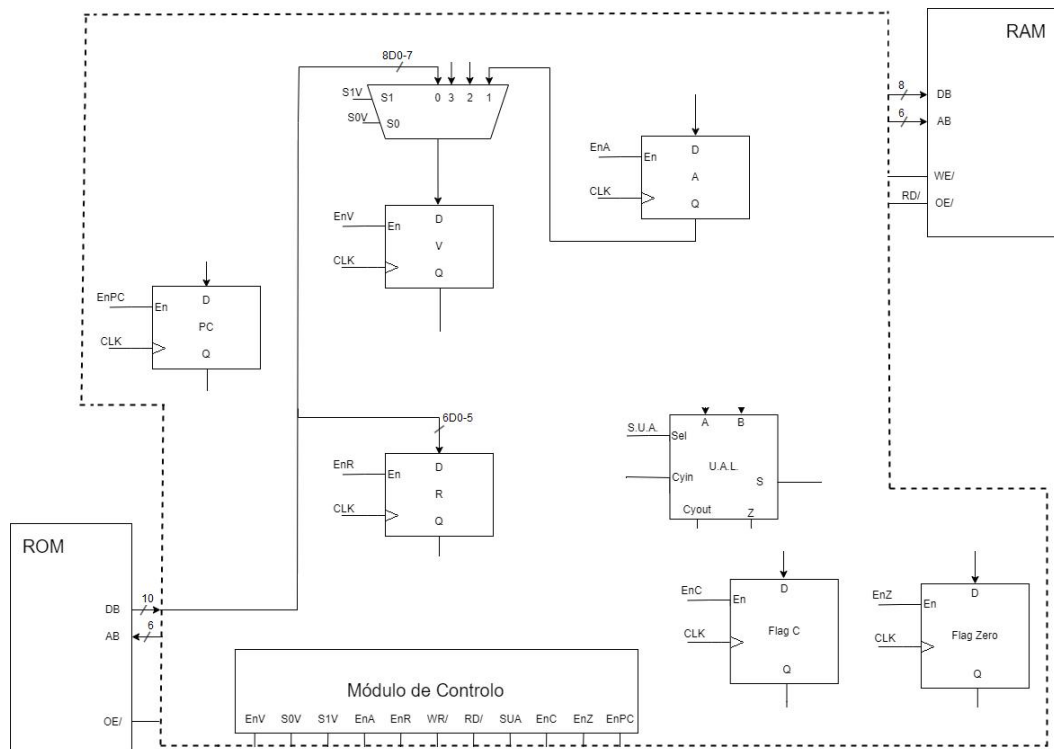


Imagem 5 - MOV V, A, V = A

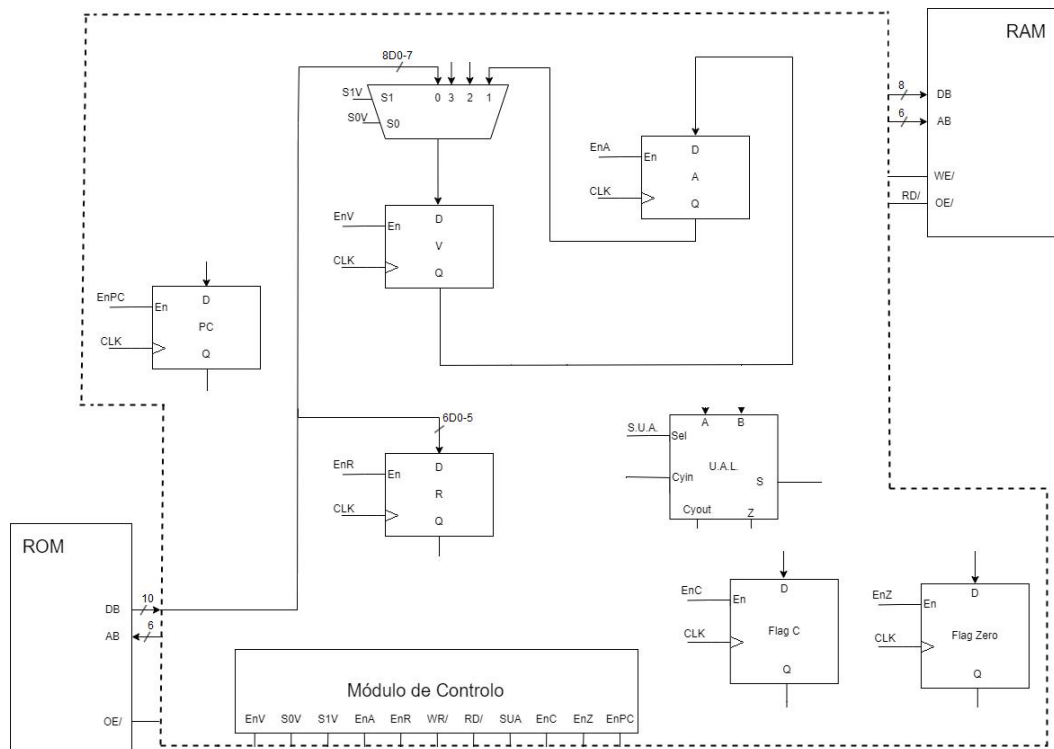


Imagem 6 - MOV A, V, A = V

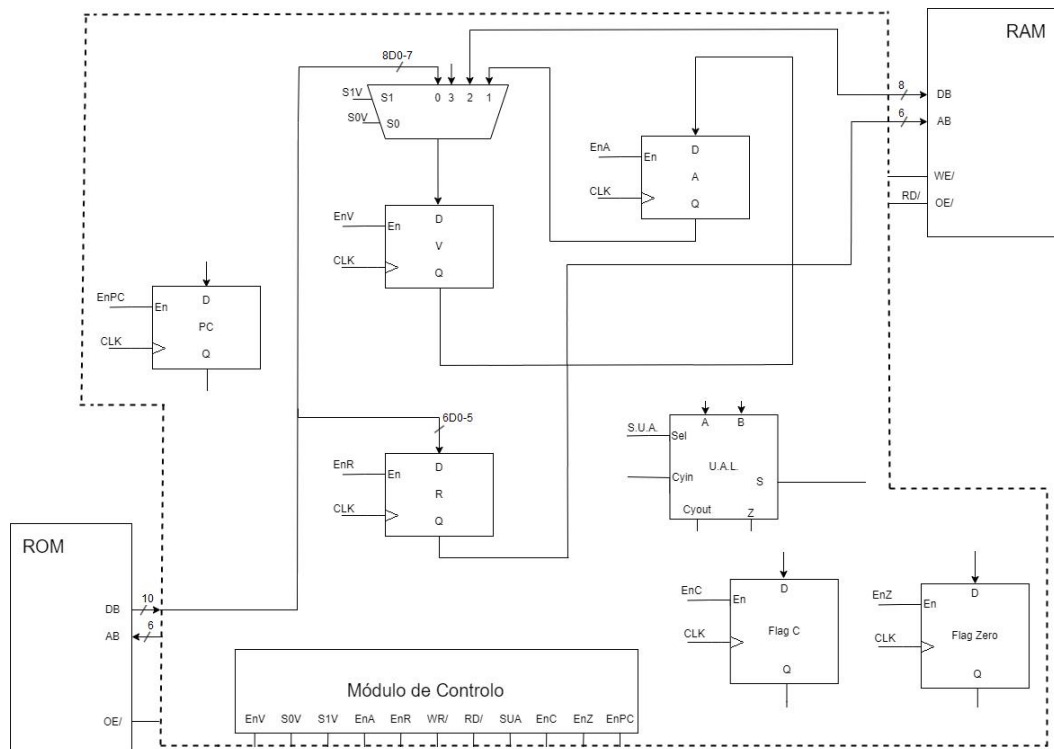


Imagem 7 - MOV V, @R, V = MD (R)

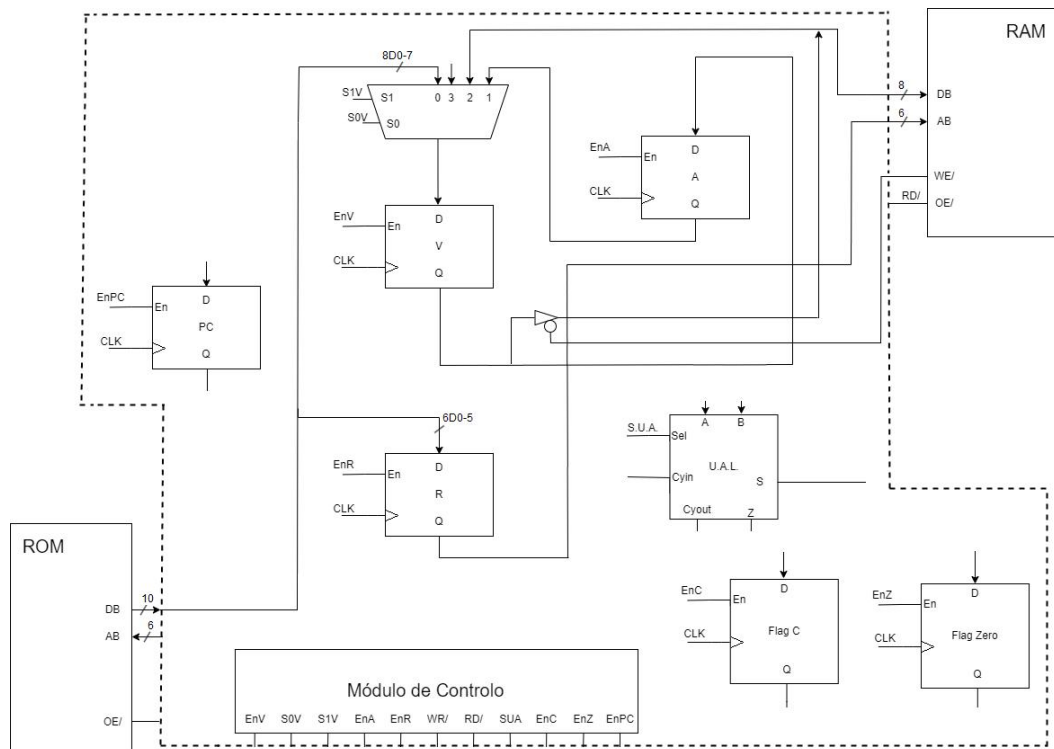


Imagem 8 - MOV @R, V MD (R) = V

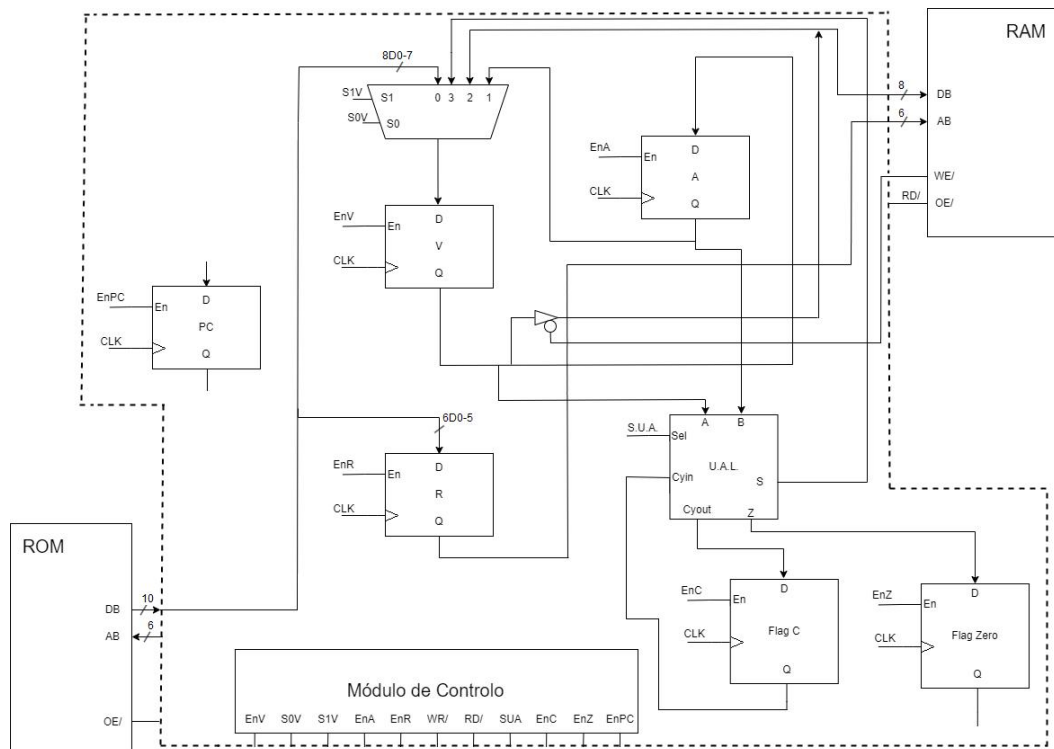


Imagem 9 - ADDC V, A $V = V + A + C$

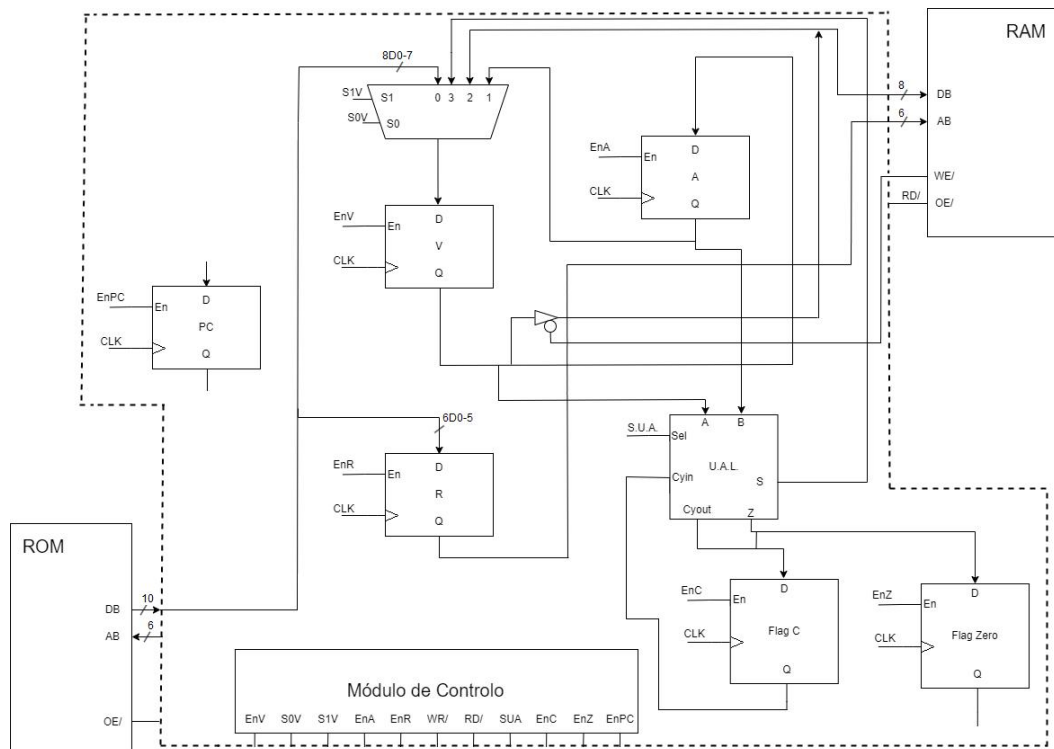


Imagem 10 - NAND V, A, $V = V \text{ nand } A$

Os bits D9, D8, D7 e D6 servem para distinguir as instruções na nossa codificação porque variam entre os valores 0 e 1.

Instrução	Parâmetros	Codificação a 10 bits									
		D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
MOV V, const8	const8	0	0	C7	C6	C5	C4	C3	C2	C1	C0
MOV R, const6	const6	0	1	0	0	K5	K4	K3	K2	K1	K0
MOV V, A		0	1	0	1	0	0	0	0	0	0
MOV A, V		0	1	1	0	0	0	0	0	0	0
MOV V, @R		0	1	1	1	0	0	0	0	0	0
MOV @R, V		1	0	0	0	0	0	0	0	0	0
ADDC V, A		1	0	0	1	0	0	0	0	0	0
NAND V, A		1	0	1	0	0	0	0	0	0	0
JC end6	end6	1	0	1	1	E5	E4	E3	E2	E1	E0
JZ end6	end6	1	1	0	0	E5	E4	E3	E2	E1	E0
JMP end6	end6	1	1	0	1	E5	E4	E3	E2	E1	E0

Tabela 4 - Codificação das instruções com os valores variantes destacados

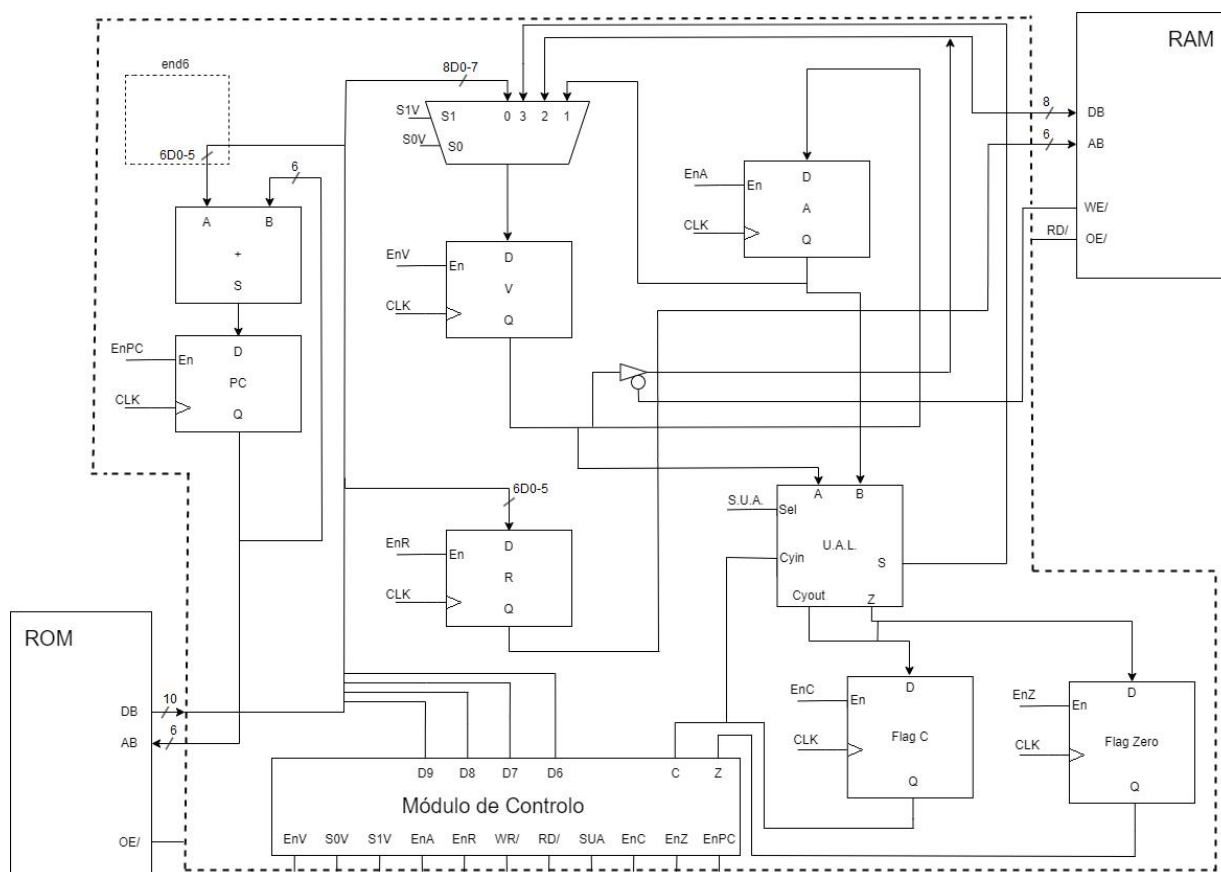


Imagem 13 - Módulo Funcional com as entradas definidas

O Módulo de Controlo tem 6 entradas o que dá uma tabela de verdades com 64 combinações possíveis.

Dividiu-se o Módulo de Controlo em módulos diferentes. Um orientado para gerar os sinais do PC, implementado com um circuito combinatório e um módulo com os restantes sinais implementado a partir de uma ROM.

Daqui tira-se a expressão para o EnPC:

$$\text{EnPC} = \text{JMP} + \text{JZ.Z} + \text{JC.C}$$

Módulo de Controlo

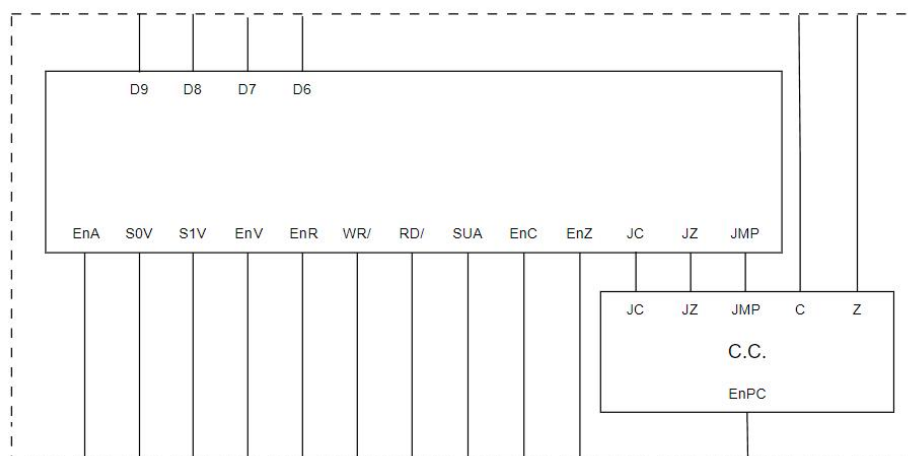


Imagem 14 - Esquema com os dois módulos distintos que formam o MC

1. 6 - Tabela de Verdade que implementa o Módulo de Controlo

Elaboraram-se duas tabelas: uma com os vários sinais ativos para cada instrução. No final, calculou-se o valor hexadecimal correspondente a cada instrução.

D9	D8	D7	D6	Sinais Ativos	SUA	EnA	S0V	S1V	EnV	EnR	WR/	RD/	EnC	EnZ	JC	JZ	JMP	Hex
0	0	X	X	EnV	0	0	0	0	1	0	1	1	0	0	0	0	0	160
0	1	0	0	EnR	0	0	0	0	0	1	1	1	0	0	0	0	0	E0
0	1	0	1	EnV, S0V	0	0	1	0	1	0	1	1	0	0	0	0	0	560
0	1	1	0	EnA	0	1	0	0	0	0	1	1	0	0	0	0	0	860
0	1	1	1	EnV, S1V, RD/	0	0	0	1	1	0	1	0	0	0	0	0	0	340
1	0	0	0	WR/	0	0	0	0	0	0	0	1	0	0	0	0	0	20
1	0	0	1	EnV, EnC, EnZ, S1V, S0V	0	0	1	1	1	0	1	1	1	1	0	0	0	778
1	0	1	0	EnV, EnZ, S1V, S0V	1	0	1	1	1	0	1	1	0	1	0	0	0	768
1	0	1	1	JC	0	0	0	0	0	0	1	1	0	0	1	0	0	64
1	1	0	0	JZ	0	0	0	0	0	0	1	1	0	0	0	1	0	62
1	1	0	1	JMP	0	0	0	0	0	0	1	1	0	0	0	0	1	61

Tabela 5 - Tabela de verdade do Módulo de Controlo

1. 7 - Tabela de programação duma ROM que implementa o Módulo de Controlo

Esta tabela é baseada na anterior, no entanto, em vez de ter uma coluna com os sinais ativos tem a gama de endereços, onde cada instrução pode ser utilizada. Na última linha, representa-se os endereços quando todos os sinais estão desativados.

A3	A2	A1	A0	Gama de Endereços	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	Hex
D9	D8	D7	D6		SUA	EnA	S0V	S1V	EnV	EnR	WR/	RD/	EnC	EnZ	JC	JZ	JMP	
0	0	X	X	[0,3]h	0	0	0	0	1	0	1	1	0	0	0	0	0	160
0	1	0	0	4h	0	0	0	0	0	1	1	1	0	0	0	0	0	E0
0	1	0	1	5h	0	0	1	0	1	0	1	1	0	0	0	0	0	560
0	1	1	0	6h	0	1	0	0	0	0	1	1	0	0	0	0	0	860
0	1	1	1	7h	0	0	0	1	1	0	1	0	0	0	0	0	0	340
1	0	0	0	8h	0	0	0	0	0	0	0	1	0	0	0	0	0	20
1	0	0	1	9h	0	0	1	1	1	0	1	1	1	1	0	0	0	778
1	0	1	0	Ah	1	0	1	1	1	0	1	1	0	1	0	0	0	768
1	0	1	1	Bh	0	0	0	0	0	0	1	1	0	0	1	0	0	64
1	1	0	0	Ch	0	0	0	0	0	0	1	1	0	0	0	1	0	62
1	1	0	1	Dh	0	0	0	0	0	0	1	1	0	0	0	0	1	61
				[E,F]h	0	0	0	0	0	0	1	1	0	0	0	0	0	60

Tabela 6 - Tabela de verdade do Módulo de Controlo com a Gama de Endereços definida

1.8 - Programas de Teste do CPU

Programa 1: Somar duas variáveis X e Y e colocar o valor da soma em Z. (com valores hexadecimais)

Endereço RAM	Dados RAM	Comentário											
1	1	X											
2	0xFF	Y											
3	60	Z											
Endereço ROM	Instrução Assembly		D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	Hex
0	MOV R, 1		0	1	0	0	0	0	0	0	0	1	0x101
1	MOV V, @R	V=X	0	1	1	1	0	0	0	0	0	0	0x1C0
2	MOV A, V	A=X	0	1	1	0	0	0	0	0	0	0	0x180
3	MOV R, 2		0	1	0	0	0	0	0	0	1	0	0x102
4	MOV V, @R	V=Y	0	1	1	1	0	0	0	0	0	0	0x1C0
5	ADDC V, A	V=Y+X	1	0	0	1	0	0	0	0	0	0	0x240
6	JC 0		1	0	1	1	0	0	0	0	0	0	0x2C0

Tabela 7 - Elaboração do Programa 1

Programa 2: Executa a operação lógica NAND em dois valores, X e Y.

Endereço RAM	Dados RAM	Comentário											
0	173	X											
1	38	Y											
2	0	Z											
3	255	W											
Endereço ROM	Instrução Assembly		D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	Hex
0	MOV R, 1		0	1	0	0	0	0	0	0	0	1	0x101
1	MOV V, @R	V=X	0	1	1	1	0	0	0	0	0	0	0x1C0
2	MOV A, V	A=X	0	1	1	0	0	0	0	0	0	0	0x180
3	MOV R, 0		0	1	0	0	0	0	0	0	0	0	0x100
4	MOV V, @R	V=Y	0	1	1	1	0	0	0	0	0	0	0x1C0
5	NAND V, A	V=Y nand X	1	0	1	0	0	0	0	0	0	0	0x280
6	MOV V, A	V=X	0	1	0	1	0	0	0	0	0	0	0x170
7	MOV R, 3		0	1	0	0	0	0	0	0	1	1	0x103
8	MOV V, @R	V=W	0	1	1	1	0	0	0	0	0	0	0x1C0
9	MOV A, V	A=W	0	1	1	0	0	0	0	0	0	0	0x180
10	NAND V, A	V=Y nand X	1	0	1	0	0	0	0	0	0	0	0x280
11	JZ 0		1	1	0	0	0	0	0	0	0	0	0x300

Tabela 8 - Elaboração do Programa 2

Programa 3: Somar duas variáveis X e Y e colocar o valor da soma em Z. (com valores decimais)

Endereço RAM	Dados RAM	Comentário											
0	8	X											
1	5	Y											
2	60	Z											
Endereço ROM	Instrução Assembly		D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	Hex
0	MOV R, 0		0	1	0	0	0	0	0	0	0	0	0x100
1	MOV V, @R	V=X	0	1	1	1	0	0	0	0	0	0	0x1C0
2	MOV A, V	A=X	0	1	1	0	0	0	0	0	0	0	0x180
3	MOV R, 1		0	1	0	0	0	0	0	0	0	1	0x101
4	MOV V, @R	V=Y	0	1	1	1	0	0	0	0	0	0	0x1C0
5	ADDC V, A	V=Y+X	1	0	0	1	0	0	0	0	0	0	0x240
6	MOV R, 2		0	1	0	1	0	0	0	0	1	0	0x102
7	MOV @R, V	@R=V	0	1	1	1	0	0	0	0	0	0	0x1C0
8	MOV V, 0		0	0	0	0	0	0	0	0	0	0	0x000
9	MOV V, @R	V=Z	0	1	1	1	0	0	0	0	0	0	0x1C0
10	JMP 0		1	1	0	1	0	0	0	0	0	0	0x340

Tabela 9 - Elaboração do Programa 3

Bibliografia

Consulta:

Mooddle:

Pais, Jorge. (2022 - 2023). *Computação Física*

Carvalho, Carlos (2023). *Computação Física*

Desenhos:

MS Paint

Draw.io - <https://app.diagrams.net/>

Código Comentado

Exercício 1:

```
//ISEL- LEIM - Grupo 06 - Computação Física - TP2 - Ex 1
//Coding UTF-08

// Definir pinos
#define pinoClk 2

//Definição das memórias
//RAM->2**8 / ROM->2**6
int RAM[256];
int ROM[64];

// Definição da ROM que implementa parcialmente o módulo de controlo
const unsigned int ROM_MC[16] = {0x160, 0x160, 0x160, 0x160, 0xE0, 0x560, 0x860, 0x340, 0x020, 0x778,
0x1768, 0x64, 0x62, 0x61, 0x60, 0x60};

// Entradas e Saídas dos registos
int DPC, QPC; // registo PC
byte DV, QV; // registo V
byte DA, QA; // registo A
int DR, QR; //registo R
bool DC, QC; // flag C
bool DZ, QZ; // flag zero

// Saidas do Módulo de Controlo
bool EnV, S0V, S1V, EnA, EnR, WRn, RDn, SUA, EnC, EnZ, JMP, JZ, JC;
bool CLK; //bool para clock

//Enable PC
bool EnPC;

void setup() {
    Serial.begin(9600);
    QPC = 0;
    loadPrograma1();
    //loadPrograma2();
    //loadPrograma3();
    pinMode(pinoClk, INPUT);
}
```

```

attachInterrupt(digitalPinToInterrupt(pinoClk), CircuitoSequencial, RISING);
interrupts();
}

void loop()
{
    moduloControlo();
    mostrarRegistos();
}

/**
 * Este método só é executado quando o clk está a true, sendo que depois esta variável é passada
para false.
 * Chama-se o metodo mostrarSinais() e faz-se print dos vários Registos
 */
void mostrarRegistos()
{
    if(CLK){
        CLK = false;
        mostrarSinais();
        Serial.print("REGISTOS: ");
        Serial.print(" PC: ");Serial.print(QPC);
        Serial.print(" QV: ");Serial.print(QV);
        Serial.print(" DV: ");Serial.print(DV);
        Serial.print(" QA: ");Serial.print(QA);
        Serial.print(" DA: ");Serial.print(DA);
        Serial.print(" QR: ");Serial.print(QR);
        Serial.print(" DR: ");Serial.print(DR);
        Serial.print(" C: ");Serial.print(QC);
        Serial.print(" Z: ");Serial.println(QZ);
    }
}

/**
 * Faz print dos vários Sinais
 */
void mostrarSinais(){
    //, EnR, WRn, RDn, SUA, EnC, EnZ, EnPC, JMP, JZ, JC;
    Serial.print("\nSINAIS: ");
    Serial.print(" EnA: ");Serial.print(EnA);
    Serial.print(" S0V: ");Serial.print(S0V);
}

```

```

Serial.print(" S1V: ");Serial.print(S1V);
Serial.print(" EnV: ");Serial.print(EnV);
Serial.print(" EnR: ");Serial.print(EnR);
Serial.print(" WRn: ");Serial.print(WRn);
Serial.print(" RDn: ");Serial.print(RDn);
Serial.print(" SUA: ");Serial.print(SUA);
Serial.print(" EnC: ");Serial.print(EnC);
Serial.print(" EnZ: ");Serial.print( EnZ);
Serial.print(" JMP: ");Serial.print(JMP);
Serial.print(" JZ: ");Serial.print(JZ);
Serial.print(" JC: ");Serial.println(JC);
}

/**
 * Neste método coloca-se o CLK a false, são chamados todas as funções que simulam os componentes
do microprocessador
 * utiliza-se duas variáveis tipo int para serem usadas entre os métodos.
 */
void CircuitoSequencial()
{
    CLK = true;
    int auxPC, auxUAL;

    //Unidade Aritmética e Lógica
    UAL8bits(SUA, QC, QV, QA, &DC, &DZ, &auxUAL);

    //Registos a 1 bit (Flag Carry e Zero)
    Registo1bit(EnC, DC, &QC);
    Registo1bit(EnZ, DZ, &QZ);

    //registro V
    mux4x8(S0V, S1V, ROM[QPC], QA, RAM[QR], auxUAL, &DV);
    Registo8bits(EnV, DV, &QV);

    //registro A
    Registo8bits(EnA, QV, &QA);

    //registro R
    Registo6bits(EnR, ROM[QPC] , &QR);

    //PC
    Somador6bits((ROM[QPC] & 0x3F), QPC, &auxPC); //int A, int B, int *S

```

```

    Contador6bits(EnPC, DPC, &QPC);

}

/**
 * Método que atualiza a variável booleana EnPC, através da expressão lógica do Módulo de Controle
 */
void CircuitoCombinatorioMC()
{
    EnPC = JMP || JZ && DZ || JC && DC;
}

/**
 * Neste método simula-se um multiplexer de duas entradas, onde se atualiza o valor de y
 * dependendo do valor do booleano S0 e faz-se uma mascara para garantir que o resultado deste é a
1 byte
 * @param S0 (bool que define o byte utilizado), dois bytes (i0 e i1) e a saída y*
 */
void mux2x8(bool S0, byte i0, byte i1, byte *y) //condição ? expressão_se_verdadeiro :
expressão_se_falso
{
    *y = S0 ? i1 : i0;
    *y &= 0xFF;
}

/**
 * Neste método simula-se um multiplexer de quatro entradas, onde se atualiza o valor de y
 * dependendo do valor do booleano S0 e S1 e faz-se uma mascara para garantir que o resultado
deste é a 1 byte.
 * Utiliza-se o método mux2x8() para simplificar o multiplexer (possivel através das variáveis
byte criadas para auxílio)
 * @param S0 e S1(bools que definem o byte utilizado), quatro bytes (i0, i1, i2 e i3) e a saída y*
 */
void mux4x8(bool S0, bool S1, byte i0, byte i1, byte i2, byte i3, byte *y)
{
    byte a,b,c;
    mux2x8(S0, i0, i1, &a);
    mux2x8(S0, i2, i3, &b);
    mux2x8(S1, a, b, &c);
    *y = c;
    *y &= 0xFF;
}

```

```

}

/**
 * Neste método simula-se a Unidade Aritmética e Lógica, dependendo do valor do booleano Sel ele
irá executar
 * uma instrução diferente (adição e nand) que envolvem dois int's a e b.
 * Atualiza-se o valor de Cyout (caso o valor de s ultrapasse 1 byte), faz-se uma máscara a s para
garantir
 * que o resultado permaneça a 1 byte e atualiza-se o valor de Z (caso s seja igual a 0)
 * @param Sel (bool que define o byte utilizado), Cyin (bool que representa o carry anterior),
dois ints (a e b) e as saídas *Cyout, *Z e *s
 */
void UAL8bits(bool Sel, bool Cyin, int a, int b, bool *Cyout, bool *Z, int *s)
{
    if(Sel)
    {
        *s = ~(a & b);
    }
    else {
        *s = (a + b + Cyin);
    }
    *Cyout = (*s & 0x100) == 0x100;
    *s =(*s & 0xFF);
    *Z =(*s == 0);
}

/**
 * Neste método simula-se um Flip flop D, que representa um registo a 8 bits onde se atualiza o
valor de Q
 * dependendo do valor do Enable
 * @param Enable (bool que define se o valor de *Q é atualizado) , dois bytes D e *Q
 */
void Registo8bits(bool Enable, byte D , byte *Q)
{
    *Q = Enable ? D : *Q; //condição ? expressão_se_verdadeiro : expressão_se_falso
}

/**
 * Neste método simula-se um Flip flop D, que representa um registo a 6 bits onde se atualiza o
valor de Q

```

```

    * dependendo do valor do Enable e faz-se uma máscara a 6 bits para garantir que o valor é sempre
a 6 bits

    * @param Enable (bool que define se o valor de *Q é atualizado) , dois ints D e *Q
    */
void Registo6bits(bool Enable, int D , int *Q)
{
    *Q = Enable ? D : *Q;
    *Q&=0x3F;
}

/**
    * Neste método simula-se um Flip flop D, que representa um registo a 1 bit onde se atualiza o
valor de Q
    * dependendo do valor do Enable
    * @param três bools Enable (bool que define se o valor de *Q é atualizado), D e *Q
    */
void Registo1bit(bool Enable, bool D , bool *Q)
{
    if(Enable)
    {
        *Q = D;
    }
}

/**
    * Neste método simula-se um somador a 6 bits, onde apenas se some o valor de A e B, atualiza-se a
saída
    * *S com esse valor e faz-se uma máscara a 6 bits para garantir que o valor é sempre a 6 bits
    * @param três ints A, B (entradas) e *S (saída)
    */
void Somador6bits(int A, int B, int *S)
{
    *S = A + B;
    *S &= 0x3F;
}

/**
    * Neste método simula-se um contador a 6 bits, onde se atualiza o valor de Q
    * dependendo do valor do Enable (true com o valor de D ou false com uma incrementação de +1 ao
seu valor )
    * e faz-se uma máscara a 6 bits para garantir que o valor é sempre a 6 bits

```

```

    * @param En (bool que define como o valor de *Q é atualizado), dois ints D e *Q
    */
void Contador6bits(bool En, int D , int *Q)
{
    if (En)
    {
        *Q = D;
    }else {
        *Q = *Q + 1; // nao houve jump
    }
    *Q &= 0x3F;
}

/**
 * Método baseado na tabela de verdade do módulo de controlo (ver tabela 5)
 */
void tabelaVerdadeMC()
{
    int addressMC = (ROM[QPC] & 0x3C0) >> 6; //11 1100 0000 -> 00 0000 1111

    JMP = (ROM_MC[addressMC] & 0x1) == 0x1;
    JZ = (ROM_MC[addressMC] & 0x2) == 0x2;
    JC = (ROM_MC[addressMC] & 0x4) == 0x4;
    EnZ = (ROM_MC[addressMC] & 0x8) == 0x8;
    EnC = (ROM_MC[addressMC] & 0x10) == 0x10;
    RDn = (ROM_MC[addressMC] & 0x20) == 0x20;
    WRn = (ROM_MC[addressMC] & 0x40) == 0x40;
    EnR = (ROM_MC[addressMC] & 0x80) == 0x80;
    EnV = (ROM_MC[addressMC] & 0x100) == 0x100;
    S1V = (ROM_MC[addressMC] & 0x200) == 0x200;
    S0V = (ROM_MC[addressMC] & 0x400) == 0x400;
    EnA = (ROM_MC[addressMC] & 0x800) == 0x800;
    SUA = (ROM_MC[addressMC] & 0x1000) == 0x1000;
}

/**
 * Método que chama a tabelaVerdadeMC() e o CircuitoCombinatorioMC()
 */
void moduloControlo(){
    tabelaVerdadeMC();
}

```

```

    CircuitoCombinatorioMC();
}

/**
 * Método do 1º enunciado de testes
 */
void loadPrograma1() //Soma de numeros
{
    //por recomendação do professor, não se utilizou o RAM no índice 0
    RAM[1] = 1;
    RAM[2] = 0xFF;
    RAM[3] = 60;

    //1MOV R, 1
    ROM[0]=0x101;

    //2MOV V, @R
    ROM[1] = 0x1C0;

    //3MOV A, V
    ROM[2] = 0x180;

    //4MOV R, 2
    ROM[3]= 0x102;

    //5MOV V, @R V=Y
    ROM[4] = 0x1C0;

    //6ADDC V, A Carry
    ROM[5] = 0x240;

    //7MOV JC 0
    ROM[6] = 0x2C0;

}

/**
 * Método do 2º enunciado de testes
 */
void loadPrograma2()
{
    RAM[0]=173; //1010 1101// 0xAD
    RAM[1]=38; //0010 0110 // 0x26

```

```

RAM[2]=0;
RAM[3]=255;

//1MOV R, 1
ROM[0]=0x101;

//2MOV V, @R
ROM[1] = 0x1C0;

//3MOV A, V A=38
ROM[2] = 0x180;

//4MOV R, 0
ROM[3]= 0x100;

//5MOV V, @R V=X V=173
ROM[4] = 0x1C0;

//6NAND V,A
ROM[5] = 0x280;

//MOV V, A
ROM[6]= 0x170;

//ativar flag do Z do NAND
//1MOV R, 3
ROM[7]=0x103;

//2MOV V, @R V=255
ROM[8] = 0x1C0;

//3MOV A, V A=255
ROM[9] = 0x180;

//6NAND V,A
ROM[10] = 0x280;

//JZ
ROM[11] = 0x300;

}

/**

```

```

* Método do 3º enunciado de testes
*/
void loadPrograma3()
{
    RAM[0] = 8;
    RAM[1] = 5;
    RAM[2] = 60;

    //1MOV R, 0
    ROM[0]=0x100;

    //2MOV V, @R
    ROM[1] = 0x1C0;

    //3MOV A, V
    ROM[2] = 0x180;

    //4MOV R, 1
    ROM[3]= 0x101;

    //5MOV V, @R V=Y
    ROM[4] = 0x1C0;

    //6ADDC V, A Carry
    ROM[5] = 0x240;

    //7MOV R, 2
    ROM[6] = 0x102;

    //8MOV @R, V RAM[R]=V /Z=V
    ROM[7] = 0x200;

    //9MOV V, 0
    ROM[8]= 0x00;

    //10MOV V, @R
    ROM[9] = 0x1C0;

    //11JMP
    ROM[10] = 0x340;
}

```